

### === SQL MANIPULATION ACTIVITY ===

--- Part 3: Create Table ---

[CREATE TABLE statement you used]

```
CREATE TABLE pokedex_test (  
  pokemon_id TEXT PRIMARY KEY,  
  name TEXT NOT NULL,  
  type1 TEXT NOT NULL,  
  type2 TEXT DEFAULT NULL  
);
```

--- Part 4: Insert Data ---

[INSERT INTO statements you used]

```
INSERT INTO pokedex_test (  
  pokemon_id, name, type1, type2  
)  
VALUES  
(1, 'Bulbasaur', 'Grass', 'Poison'),  
(2, 'Charmander', 'Fire', NULL),  
(3, 'Charizard', 'Fire', 'Flying'),  
(4, 'Pikachu', 'Electric', NULL),  
(5, 'Gyarados', 'Water', 'Flying');
```

[Output of SELECT \* FROM pokedex\_test after inserting]

```
[sqlite> SELECT * FROM pokedex_test;  
pokemon_id  name          type1         type2  
-----  
1           Bulbasaur    Grass        Poison  
2           Charmander   Fire  
3           Charizard    Fire         Flying  
4           Pikachu     Electric  
5           Gyarados     Water        Flying  
[sqlite> UPDATE pokedex_test
```

--- Part 5: Update Data ---

[UPDATE statement you used]

```
UPDATE pokedex_test
SET name = 'Charilizardo'
WHERE pokemon_id = 3;
```

[Output of SELECT \* FROM pokedex\_test after updating]

```
[sqlite> SELECT * FROM pokedex_test;
pokemon_id  name          type1      type2
-----
1           Bulbasaur    Grass      Poison
2           Charmander   Fire
3           Charilizardo Fire      Flying
4           Pikachu     Electric
5           Gyarados    Water      Flying
```

--- Part 6: Delete Data ---

[DELETE statement you used]

```
DELETE FROM pokedex_test
WHERE pokemon_id = 3;
```

[Output of SELECT \* FROM pokedex\_test after deleting]

```
sqlite> SELECT * FROM pokedex_test;
pokemon_id  name          type1      type2
-----
1           Bulbasaur    Grass      Poison
2           Charmander   Fire
4           Pikachu     Electric
5           Gyarados    Water      Flying
```

--- Part 7: Constraint Exploration ---

[INSERT statement that violated the identifier constraint + error output]

```
INSERT INTO pokedex_test (  
pokemon_id, name, type1, type2  
)  
VALUES  
(1, 'Ivysaur', 'Water', 'Flying')
```

```
sqlite> INSERT INTO pokedex_test (  
(x1...>     pokemon_id, name, type1, type2  
(x1...> )  
    ...> VALUES  
[    ...> (1, 'Ivysaur', 'Water', 'Flying')  
[    ...>  
[    ...> ;  
Runtime error: UNIQUE constraint failed: pokedex_test.pokemon_id (19)  
sqlite> INSERT INTO pokedex_test (  
(x1...>     pokemon_id, name, type1, type2  
(x1...> )  
    ...> VALUES  
[    ...> (1, 'Ivysaur', 'Water', 'Flying')  
[    ...>  
[    ...> ;  
Runtime error: UNIQUE constraint failed: pokedex_test.pokemon_id (19)  
sqlite>
```

[INSERT statement that violated the NOT NULL constraint + error output]

```
INSERT INTO pokedex_test (  
pokemon_id, name, type1, type2  
)  
VALUES  
(10, NULL, 'Water', 'Flying')
```

```
sqlite> INSERT INTO pokedex_test (  
(x1...>     pokemon_id, name, type1, type2  
(x1...> )  
    ...> VALUES  
[    ...> (10, NULL, 'Water', 'Flying')  
[    ...>  
[    ...> ;  
Runtime error: NOT NULL constraint failed: pokedex_test.name (19)  
sqlite>
```

--- Post-activity ---

[Output of SELECT \* FROM pokedex\_test]

```
sqlite> SELECT * FROM pokedex_test;
pokemon_id  name          type1         type2
-----
1           Bulbasaur    Grass        Poison
2           Charmander   Fire
4           Pikachu     Electric
5           Gyarados    Water        Flying
sqlite> █
```

--- Reflection Questions ---

1. When inserting a duplicate value in the identifier column, an error message displays "Runtime error: UNIQUE constraint failed: pokedex\_test.pokemon\_id (19)" in the terminal. This is because of the PRIMARY KEY constraint added into the column id, hence it is impossible to have duplicate id's in the table.
2. When inserting a record with a missing name value, the error message shows "Runtime error: NOT NULL constraint failed: pokedex\_test.name (19)". This is because a NOT NULL constraint is placed in the column name, hence every record must have a value for name, and it cannot be left empty or NULL
3. Constraints are very important when designing these database tables because having constraints are like having rules for your created table. This protects the database from incorrect, missing, and duplicate data. Also, this makes the table organized and reliable which makes it easy to search, update, and use.